

Toegepaste Informatica

2TX

2025-2026



UCLL
HOGESCHOOL



Full-stack software development

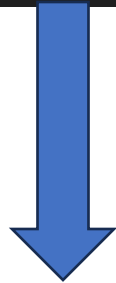
Front-end: Server components

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

CreateSchedulePage: adding a useState hook

- When we add a useState hook, the compiler will complain

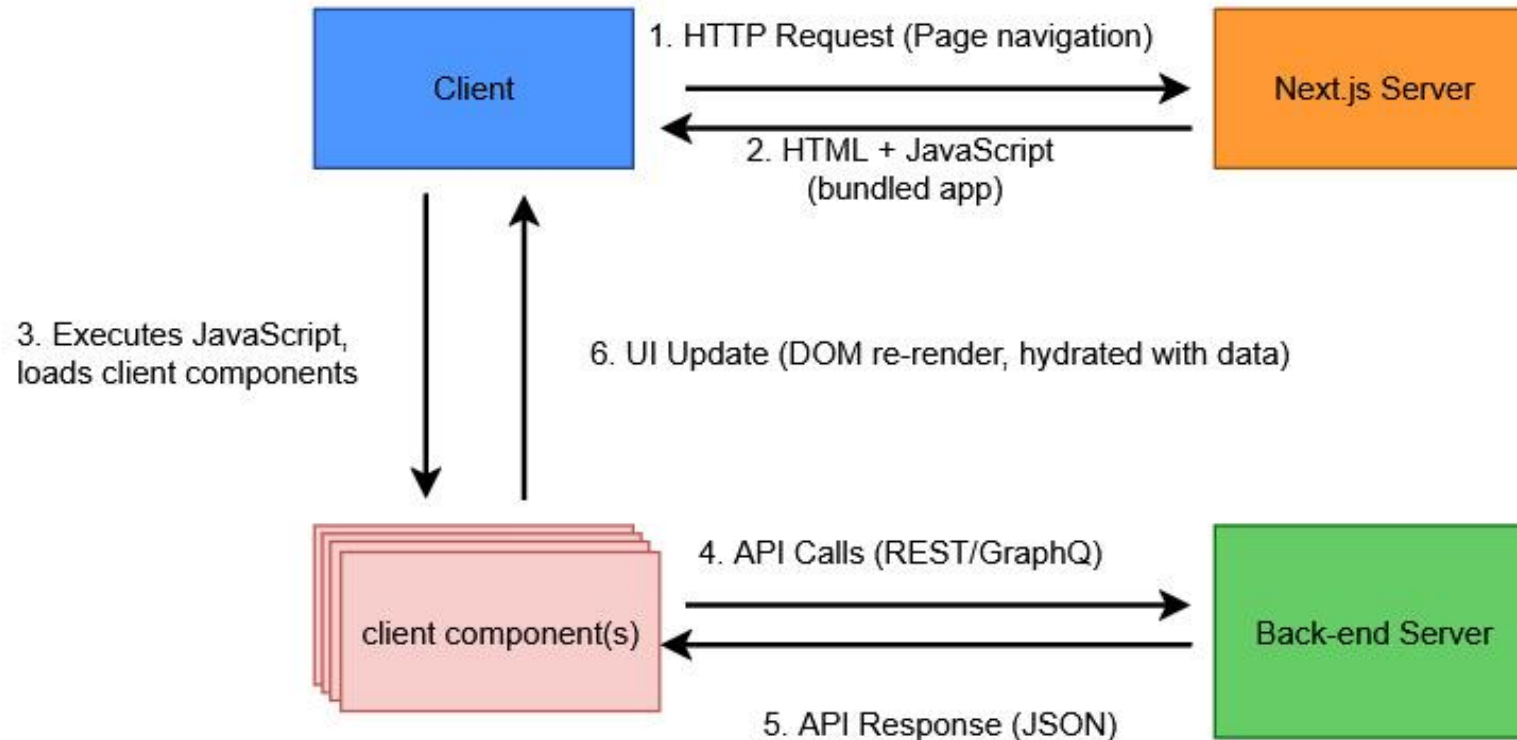
```
// This line will cause a compiler error:  
const [lecturerName, setLecturerName] = useState<string | null>(null);
```



```
Import trace for requested module:  
./app/schedule/[lecturerId]/[courseId]/page.tsx  
✖ ./app/schedule/[lecturerId]/[courseId]/page.tsx  
Error: ✖ You're importing a component that needs `useState`. This React Hook only works in a Client Component. To fix, mark the file (or its parent) with the `"use client"` directive.  
  
Learn more: https://nextjs.org/docs/app/api-reference/directives/use-client
```

Traditional React component

- Up to now we have used traditional react components. These are rendered on the client:



CreateSchedulePage is a server component

```
const CreateSchedulePage = async ({ params }: PageProps) => {  
  const paramsResolved = await params;  
  const { data, error } = await getData(  
    paramsResolved.lecturerId,  
    paramsResolved.courseId  
  );  
  
  return (  
    <>  
      <h1>Create new schedule</h1>  
  
      {error && (  
        <div className="text-red-800" role="alert">  
          {error}  
        </div>  
      )}  
  
      {data && (  
        <section>  
          <ScheduleForm lecturer={data?.lecturer} course={data?.course} />  
        </section>  
      )}  
    </>  
  );  
};  
  
export default CreateSchedulePage;
```

Page is an **async** function

await the fulfillment of the params promise to get the resolved parameters.

Why? This component is rendered on the server, and its data dependencies (like route params) may not be immediately available. This is especially true if the params are derived from dynamic routes.

await the result of the get data function and destructure the result.

CreateSchedulePage is a server component

- This component is created on the Next.js server and send to the client as static html
- Can we prove this with this stripped-down version of the component?

```
type PageProps = {
  params: Promise<{
    lecturerId: string;
    courseId: string;
  }>;
};

const CreateSchedulePage = async ({ params }: PageProps) => {
  const paramsResolved = await params;

  console.log('paramsResolved:', paramsResolved);

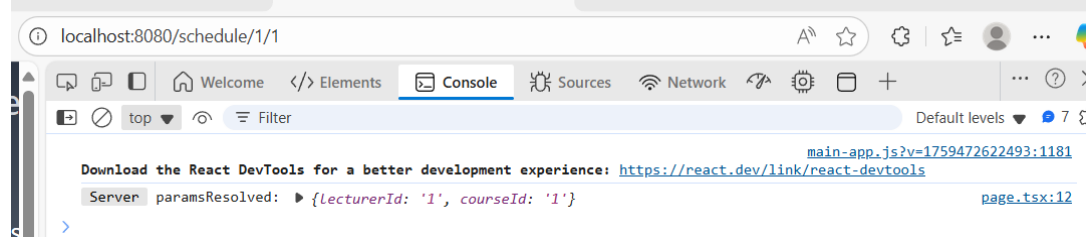
  return (
    <>
      <h1>Create new schedule</h1>
    </>
  );
};

export default CreateSchedulePage;
```

Once the params are resolved, we log them to the console. We expect to find this only in our browser's console.

CreateSchedulePage is a server component

```
✓ Starting...
✓ Ready in 1566ms
  ○ Compiling /schedule/[lecturerId]/[courseId] ...
✓ Compiled /schedule/[lecturerId]/[courseId] in 1783ms (596 modules)
✓ Compiled in 428ms (234 modules)
paramsResolved: { lecturerId: '1', courseId: '1' }
GET /schedule/1/1 200 in 3120ms
```

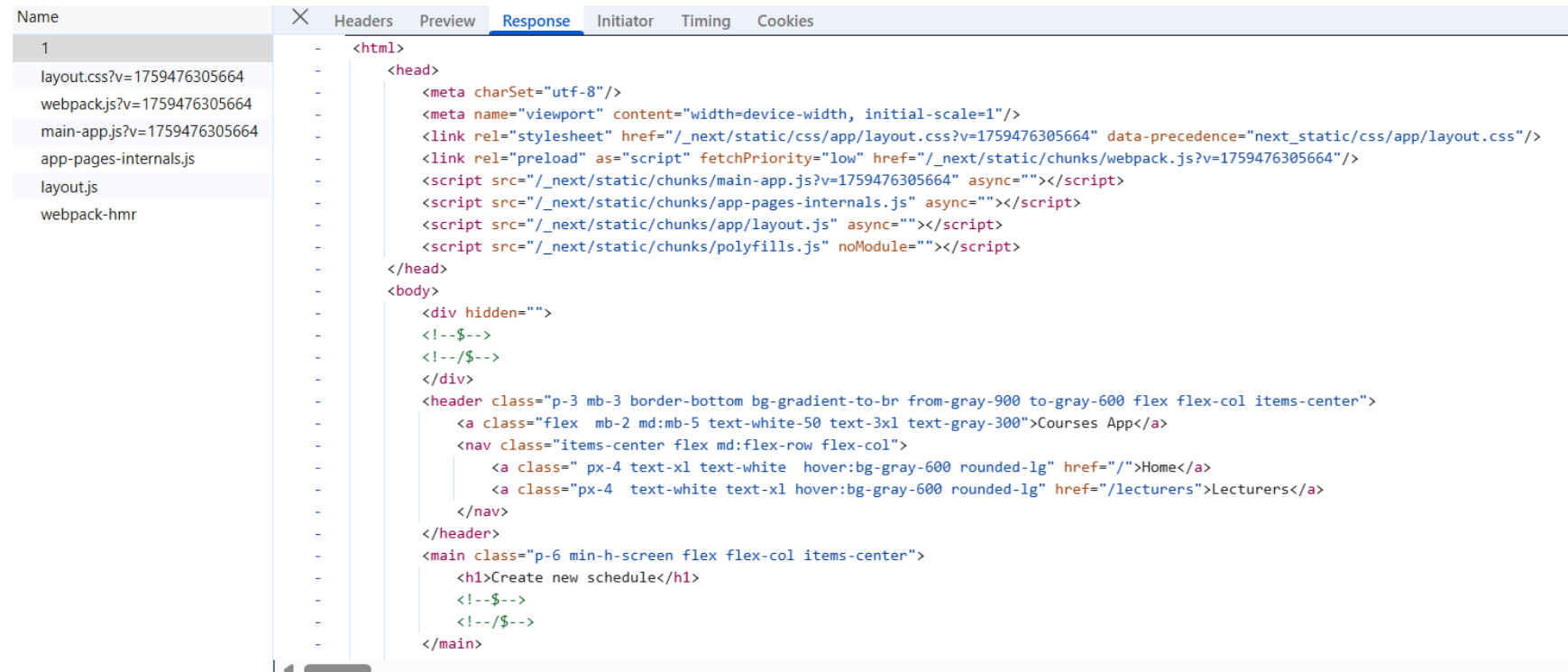


We find the log

- in the output of our server (in the terminal of VSCode)
- In our dev tools console, but marked as “server” output (only in development, not in production code)

CreateSchedulePage is a server component

And it was returned as HTML. We can check that in the network tab or dev tools:



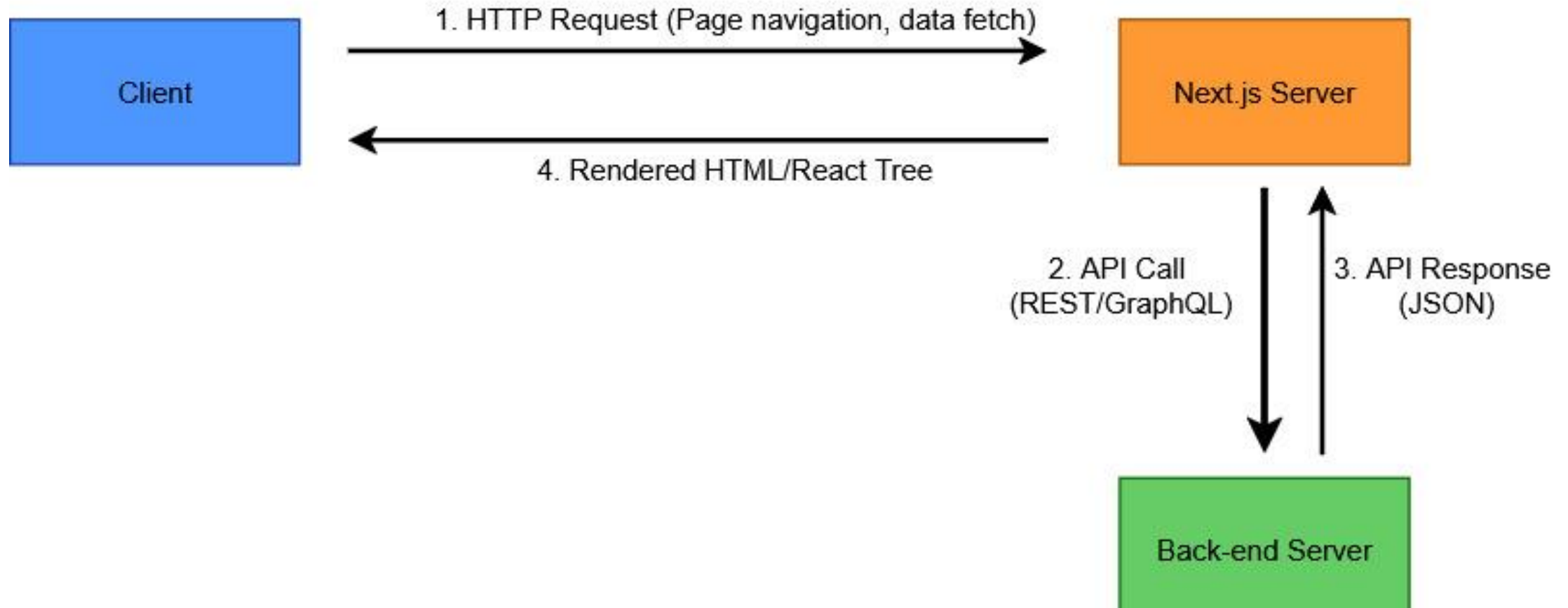
The screenshot shows the 'Response' tab of a browser's developer tools. On the left, a list of network requests is shown, with the first one selected. The main pane displays the HTML response of this request. The HTML structure includes a head section with meta tags, viewport settings, and various CSS and JavaScript links. The body section contains a header with navigation links for 'Courses App', 'Home', and 'Lecturers', followed by a main content area with the heading 'Create new schedule'.

```
<html>
<head>
  <meta charset="utf-8"/>
  <meta name="viewport" content="width=device-width, initial-scale=1"/>
  <link rel="stylesheet" href="/_next/static/css/app/layout.css?v=1759476305664" data-precedence="next_static/css/app/layout.css"/>
  <link rel="preload" as="script" fetchPriority="low" href="/_next/static/chunks/webpack.js?v=1759476305664"/>
  <script src="/_next/static/chunks/main-app.js?v=1759476305664" async=""></script>
  <script src="/_next/static/chunks/app-pages-internals.js" async=""></script>
  <script src="/_next/static/chunks/app/layout.js" async=""></script>
  <script src="/_next/static/chunks/polyfills.js" noModule=""></script>
</head>
<body>
  <div hidden="">
    <!--$-->
    <!--/$-->
  </div>
  <header class="p-3 mb-3 border-bottom bg-gradient-to-br from-gray-900 to-gray-600 flex flex-col items-center">
    <a class="flex mb-2 md:mb-5 text-white-50 text-3xl text-gray-300">Courses App</a>
    <nav class="items-center flex md:flex-row flex-col">
      <a class="px-4 text-xl text-white hover:bg-gray-600 rounded-lg" href="/">Home</a>
      <a class="px-4 text-white text-xl hover:bg-gray-600 rounded-lg" href="/lecturers">Lecturers</a>
    </nav>
  </header>
  <main class="p-6 min-h-screen flex flex-col items-center">
    <h1>Create new schedule</h1>
    <!--$-->
    <!--/$-->
  </main>
```

Server-side and client-side rendering

- A restaurant analogy:
 - Server-side rendering is like eating at a traditional restaurant
 - The cook prepares everything in the kitchen, plates it up and it gets delivered to your table completely ready to eat.
 - Client-side rendering is like eating at a fondue restaurant
 - A waiter will bring all ingredients to the table, but the client must assemble everything him/herself.

Server components



Server components: benefits

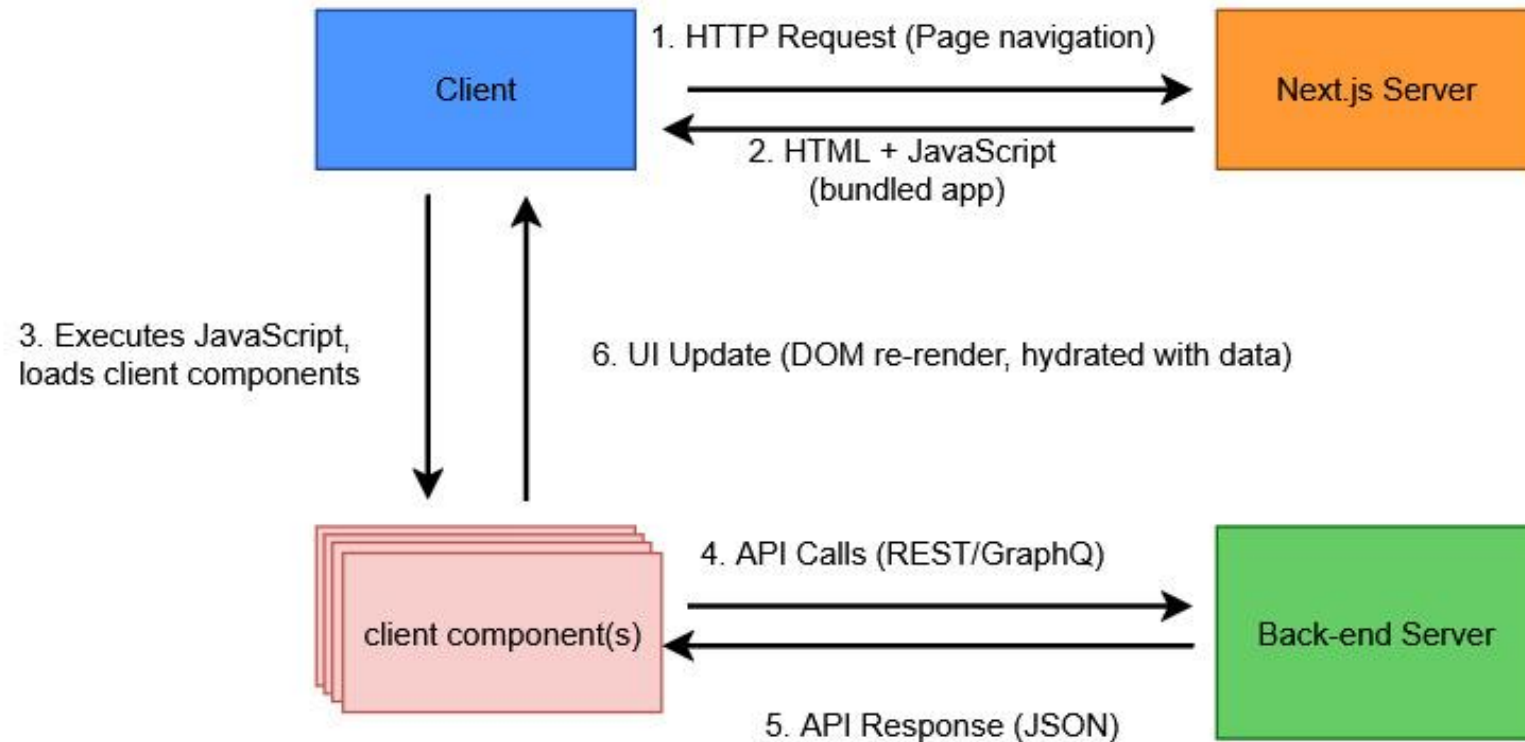
- This is the default component type (starting at Nextjs version 13)
- Components are created (including fetched data) on the server, no JavaScript needs to be sent to the front-end.
 - Faster data fetching
 - Smaller packages
 - Faster time to First Meaningful Paint (the moment a front-end becomes useable by a user after a page load)
- Can render client components as children (see below)

Server components: limitations

- No client-side JavaScript, so we can't use hooks or state
- Server components have no access to Browser APIs (like cookies and local storage)

Client components

- When working with Next.js, the “old” react components are called client components:



Client components

- To mark a component as a client component, add this directive at the top of your file: 'use client';
- Use a client component if you need:
 - Interactivity: button clicks, forms,...
 - Hooks
 - Browser APIs (like sessionStorage)
- These are the kind of components we used in lab-02

Best practice

mix and match Server and Client Components to leverage the strengths of both environments:

- Server components render the base of your UI,
- Client components can be used to add interactivity where needed

Fetching data: server components

```
const Page = async () => {  
  const data = await fetch('https://api.vercel.app/blog')  
  const posts = await data.json()  
  return (  
    <ul>  
      {posts.map((post) => (  
        <li key={post.id}>{post.title}</li>  
      ))}  
    </ul>  
  )  
}
```

Using the fetch api.

- Fetch is asynchronous so it needs await
- The component needs to be made async
- The .json method is also async and needs await

Fetching data: server components

```
type DataResult = {  
  data: Lecturer[] | null;  
  error: string | null;  
};  
  
const getData = async (): Promise<DataResult> => {  
  try {  
    const lecturers = await LecturerService.getAllLecturers();  
  
    return { data: lecturers, error: null };  
  } catch (error) {  
    return { data: null, error: (error as Error).message };  
  }  
};  
  
const LecturersPage = async() => {  
  const { data, error } = await getData();  
  
  return (  
    <>  
    <h1>Lecturers</h1>  
  )  
};
```

```
const getAllLecturers = async (  
) : Promise<Lecturer[]> => {  
  const response = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/lecturers`, {  
    method: 'GET',  
    headers: {  
      'Content-Type': 'application/json',  
    },  
    cache: 'no-store',  
  });  
  
  if (!response.ok) {  
    const error = await response.json().catch(() => ({}));  
  }  
  
  return response.json();  
};
```

Best practice:

Encapsulate all the fetch functionality in a service and call that from the server component

Fetching data: client components

The 3 ways to fetch data in client components, all use hooks

- The use hook to stream data from a server component to a client component => this goes beyond the scope of this course. You can learn more using the references at the end.
- The useSWR hook: a third-party hook. Preferred for data fetching.

useSWR

- Is a custom hook provided by the swr library from Vercel that helps handle data fetching and caching in client components
- It uses a technique called **Stale-While-Revalidate** (SWR), where it returns the cached data immediately and then fetches updated data in the background.

```
npm install swr
```

- <https://nextjs.org/docs/pages/building-your-application/data-fetching/client-side>
- <https://blog.logrocket.com/handling-data-fetching-next-js-useswr/>

useSWR

- **Stale:** useSWR immediately returns the cached data (if any) to the user, providing fast rendering.
- **While:** While the user views the data, useSWR starts fetching the most recent data in the background.
- **Revalidate:** Once the new data is retrieved, the component is re-rendered and the latest data is displayed.

useSWR

```
const { data, error, isLoading, isValidating, mutate } = useSWR(key, fetcher, options)
```

- useSWR expects a
 - **key**: a unique string, for data fetching use the url, is used to identify the data that is being fetched and cached.
 - **fetcher** function: an async function that fetches the data
 - Options: (*optional*) an object of options for this SWR hook
- useSWR returns:
 - **data**: the result of the fetcher function, the fetched data
 - **Error**: If the fetcher function throws an error, it is stored here.
 - **isLoading**: a Boolean indicating the progress of the fetch request
 - A **mutate** function to trigger a re-evaluation of the cache (see below)
 - **IsValidating**: a Boolean indicating that useSWR is re-evaluating its cache.

useSWR: an example

```
const fetcher = () => ScheduleService.getSchedules();

const ScheduleOverviewClientPage = ({
  initialSchedules,
  allStudents,
}: Props) => {
  const { user } = useAuth();

  const { data: schedules, mutate } = useSWR('schedules', fetcher, {
    fallbackData: initialSchedules || undefined,
    refreshInterval: 5000,
  });
};
```

References

- <https://nextjs.org/learn/react-foundations/server-and-client-components>
- <https://vercel.com/blog/understanding-react-server-components>
- Learn more (optional) [Data streaming in react](#)